

rse_annotations: Forschungscode markieren und prüfen

Ein Werkzeug für Responsible RSE

Gesellschaft für Informatik deRSE Julian Dehne Florian Goth Jan
Phillip Thiele Jan Linxweiler Anna-Lena Lamprecht Maja Toebs

Idee

- Forschungscode wird zunehmend generiert. Niemand liest tausende Zeilen plausibel aussehenden Codes vollständig.
- **Idee:** Wer den Code schreibt, *markiert* die Stellen, an denen das wissenschaftliche Ergebnis falsch werden kann.
- Werkzeuge (**Plugins**) nutzen die Markierungen, um einen Menschen gezielt dorthin zu führen.

Die Maschine schlägt vor, der Mensch entscheidet.

Ordner	Aufgabe
decorators/	die Markierungen selbst
scan/	statische Analyse, importiert nichts
inspection/	menschliche Prüfung, Urteile
core/	Plugin-Vertrag, Zielprojekt, Befunde
plugins/hazards/	ein Ordner pro Responsible-RSE-Frage

Decorators

Was ist ein Decorator?

```
@functional  
def normalize(v, lo, hi):  
    return (v - lo) / (hi - lo)
```

- **Bedeutet:** `normalize = functional(normalize)`.
- Läuft **einmal, beim Import** – nicht bei jedem Aufruf.
- Beim Import: Eintrag ins Register. Beim Aufruf: nichts Zusätzliches.

Definitionszeit vs. Aufrufzeit

- In Python führt ein Import Code aus (anders als in Java).
- **Java-Annotationen** sind reine Metadaten, geprüft vom Compiler.
- **Python-Decorators** sind ausführbare Funktionen.

Ein neuer Decorator = eine Funktion + ein Eintrag in DECORATORS. Der Hilfetext ist der Docstring.

Jeder Decorator markiert eine Stelle, an der das Ergebnis falsch werden kann, und sagt, worauf man prüfen soll:

- @functional: Mathematik
- @mapping: Umformung von Daten
- @data_input / @data_output: Datenein- und -ausgang
- ... die Menge ist offen und wächst.

AOP-Bild: Concern = was prüfen; Join Point = die markierte Funktion; Aspekt = das Plugin, das prüft.

Inspection

- Quelltext und abgeleitete **Formel** zeigen → ja / nein / später.
- Eine Zeile Mathematik gegen die eigene Absicht prüfen. Gerendert, nicht bewiesen.
- `inspection.yaml` hält nur **menschliche Urteile**: “jemand hat das geprüft” wird nachvollziehbar.
- **Kein LLM prüft LLM-Code.** Bewusste Grenze des Werkzeugs.

Maschinelle Prüfung $\neq$ menschliches Urteil

- Automatische Checks liefern **Befunde**: Ist `@functional` wirklich rein? Liest `@data__input` wirklich?
- Nur das Urteil eines Menschen wird als **Verdict** festgehalten.
- Beides getrennt halten – auch im Code.

Plugins

Ein Plugin = eine Frage

- Lizenz, Dual Use, Footprint, ... – jede Responsible-RSE-Frage ist ein Plugin.
- Dieselben Modi für alle: **Inspektion, Analyse, Testgenerierung**.
- Neue Frage = neues Plugin, kein Umbau des Frameworks.
- Ein **Stub** ist ein Plugin ohne Implementierung; sein Docstring ist die Aufgabe für Studierende.

Klassen sind Objekte

- Das Register ist ein Tupel von **Klassen**, nicht von Instanzen.
- Metadaten (name, question, tier) stehen als `ClassVar` an der Klasse – lesbar, ohne etwas zu instanziiieren.
- Java-Vergleich: eher `static final + Class<?>`.

Nicht implementiert $\neq$ Fehler. Ein Stub meldet `available() = False` und erscheint als *skipped* mit Begründung. Der Lauf bricht nicht ab, die Lücke bleibt sichtbar.

Scan

Wie der Scan arbeitet

1. Alle *.py-Dateien finden (Tests, Caches, Vendored überspringen).
2. Jede Datei mit `ast.parse` in einen Baum verwandeln – nichts wird importiert oder ausgeführt.
3. Import-Aliase auflösen (`@f` aus `import functional as f` zählt als `@functional`).
4. Jede Funktion besuchen: Zählt sie mit? Welcher Decorator? Falls keiner: Vorschlag. Welche Hazards?
5. Hazards den Aufrufgraph hinauf vererben (wer eine LLM-Funktion aufruft, erbt `model_call`).
6. Ergebnis: `CoverageReport` – Abdeckung + Kandidatenliste.

Vorschlag aus der Form des Rumpfs

- liest *und* schreibt → aufteilen
- schreibt → @data_output; liest → @data_input
- nur Arithmetik → @functional
- Eingabe rein, anderer Wert raus → @mapping
- gibt nichts zurück → Glue-Code

Immer mit Konfidenz und Begründung: **eine Arbeitsliste, kein Urteil.**

Refactoring-Lehrstücke

Magische Strings → ein Enum

- "functional" stand verstreut in Decorators, Scan und Plugins. Ein Tippfehler fällt erst zur Laufzeit auf.
- Jetzt: HazardDecorator.FUNCTIONAL. Text nur noch an den Rändern: AST, YAML, JSON.

```
class HazardDecorator(str, Enum):  
    FUNCTIONAL = "functional"  
    MAPPING = "mapping"  
    ...
```

- Ein Enum, das zugleich ein String ist: direkt serialisierbar.

Komposition statt Vererbung

- Ein Hazard *verfeinert* typischerweise einen Decorator (statistical → @functional), ist aber keine Unterklasse davon.
- compute__dimension__icr ist @mapping **und** statistical. Vererbung würde einen Widerspruch erzwingen.
- Darum eine Referenz, keine Oberklasse:

```
HazardKind(name="statistical",  
           parent=HazardDecorator.FUNCTIONAL, ...)
```

- Zwei Achsen, unabhängig.

- Jede HazardKind trägt ihren Detektor als Funktion:
`detect=__detect_stochastic.`
- Neuer Hazard = eine Funktion + ein Eintrag in HAZARD_KINDS. Dasselbe Prinzip wie bei DECORATORS.
- Die Tabellen (HAZARDS, HAZARD_PARENT, ...) werden daraus **abgeleitet**, nie von Hand gepflegt.

- Welche Namen als Evidenz zählen (openai, shuffle, cohen_kappa_score, ...) steht in scan/vocabulary.py: reine Daten.
- Neue Bibliothek beibringen = Liste erweitern; der Code bleibt unverändert.
- Die Heuristik wird prüfbar: Man sieht, *woran* der Scan glaubt.

Ein Test hält die Quellen synchron. HazardDecorator und DECORATORS müssen dieselben Namen tragen. Ein Test prüft das, statt es zu hoffen.

Reflexion

- Wo endet, was ein Werkzeug prüfen kann, und wo beginnt Verantwortung?
- Wer trägt die Verantwortung für ein Ergebnis, dessen Code niemand vollständig gelesen hat?
- [TODO] Diskussionsfrage für die Gruppe